

Developing Theory and Methods to Understand and Improve Collaboration in Open Source Software Development on GitHub

Olivia B. Newton, Stephen M. Fiore, and Jihye Song
University of Central Florida

As a result of fundamental changes in organizational needs and practices, social coding, a facet of modern collaboration, has become a prevalent phenomenon in software development. While the adoption of social media platforms for social coding, like GitHub, has enabled distributed, asynchronous collaboration among software developers, the structure of such platforms introduces a novel set of socio-technical challenges that cognitive engineering is uniquely qualified to address. Towards this end, we examine GitHub's social and technical features as a means for both improving and hindering coordination and problem solving in software development. Through an integration of existing theories from the organizational sciences with recent research in social coding, we offer a set of preliminary research questions aimed at improving understanding of open source software development.

INTRODUCTION

Collaboration as a method of work has continued to increase in the modern organization. As problems become more complex, as innovation becomes a primary driver for change, and as the need for complementary forms of knowledge and skills increases, the modern workforce demands collaboration and teamwork (Fiore et al., 2010). Concomitant with this, though, is a need to develop collaborative infrastructures to scaffold this work (Fiore & Wiltshire, 2016). To support this, a number of organic support systems (e.g., GitHub), or industry-based systems (e.g., BaseCamp), have emerged. From the practical standpoint, at issue is the degree to which such systems enable or hinder collaboration. From the theoretical standpoint, researchers must address how these systems are altering the nature of work. The field of cognitive engineering, grounded in socio-technical systems thinking, is well positioned to address these questions. In this paper we discuss how social coding, an important facet of modern collaboration, can be examined through the lens of cognitive engineering.

In the last decade, the social coding paradigm has become increasingly common in software development and, as a result, gained attention from researchers interested in social and technical factors influencing social coding processes and outcomes (Goggins, Mascaro, & Valetto, 2013; Yu, Yin, Wang, & Wang, 2014). Social coding as a prevalent phenomenon emerged from changes in organizational practices for software engineering, namely the popularity of agile and distributed development approaches and evolution of software business (Abrahamsson, Salo, Ronkainen, & Warsta, 2002; Dingsøyr, Nerur, Balijepally, & Moe, 2012). In combination with version control and task management systems, this has played a critical role in fostering fundamental shifts in how software developers engage in collaborative projects. Further, because of the adoption of social media platforms aimed at supporting social coding, the development of software is often temporally and geographically distributed, allowing developers to work together in real time or asynchronously (Yu et al., 2014).

Social coding platforms, like GitHub, are critical for the success of open source software development (OSSD) because they support collaboration among developers working together on large-scale software projects. GitHub, in particular, is used

to host software projects belonging to major companies like Microsoft and Facebook (Octoverse, 2018). Additionally, as of 2013, the number of software developers on GitHub was reported to be over 3.5 million (Yu et al., 2014; Reyes López, 2017). The widespread adoption of social coding platforms has generated a novel problem space for researchers. As such, we suggest that the distributed nature of work in social coding introduces a set of socio-technical challenges that cognitive engineering is uniquely qualified to address.

We focus on one social coding platform, GitHub, to illustrate how research can inform specific aspects of modern work. In platforms like GitHub, OSSD is typically characterized by collaborative work between a small team of paid developers and a larger, distributed group of volunteers (McDonald & Goggins, 2013). Additionally, in OSSD, rather than work on tasks assigned to them by project leadership, many developers choose tasks based on their personal interests, a desire to build a reputation with their peers, and other socio-political drivers. Finally, in OSSD, the line between developer and customer is often blurred; volunteer developers are often consumers who use the software for their personal or occupational needs (Abrahamsson et al., 2002).

SOCIAL CODING AS A SOCIO-TECHNICAL SYSTEM

As a component of modern socio-technical systems, platforms like GitHub are a relatively new and significant means of supporting social interaction, organizational productivity, and technological advancement. In line with studies of collaboration in science and technology, such work relies on both social and job relevant factors for goal completion. More specifically, this work requires an effective integration of both teamwork and taskwork (Fiore, Carter, & Asencio, 2015; Fiore & Wiltshire, 2016). As a tool for OSSD, GitHub is designed to support teamwork and task completion throughout the development process. For instance, GitHub allows its users to track the activities of other users involved in software projects that interest them. GitHub is also designed to help connect users with shared interests and facilitate resulting collaborations. Through these mechanisms, GitHub is expected to help individuals and organizations improve their productivity and the quality of project outcomes.

The diversity of the work done in GitHub makes an analysis of team cognitive processes challenging. Nonetheless, we can examine it from a task analytic perspective to identify the socio-technical features supporting team functioning. In the extant literature, the structures of GitHub are typically broken down into users, repositories, and features (Reyes López, 2017). The interaction of these forms the foundation for GitHub as a socio-technical system for OSSD.

- **Users.** GitHub currently allows both organizations and individuals to create accounts. An organization, like Microsoft, can create an organizational account and an individual can create a personal user account.
- **Repositories.** Individuals users and organizations can host projects on GitHub through the use of repositories. While not all GitHub repositories are dedicated to software development, the current paper applies only to repositories used for OSSD. In OSSD, these repositories are public and are used to host, for example, source code and documentation. Multiple repositories may form a larger project ecosystem produced by a community of developers.
- **Features.** GitHub has several features for social interaction. Users can “follow” other users to receive notifications about what those users are doing, both across and within repositories. In terms of user-repository interaction, users can create (and delete), fork (i.e., create a personal copy), watch, or star a repository. If a user chooses to watch a repository, they will receive detailed notifications about events occurring within the repository. The star feature, introduced in 2012, adds the repository to a favorites list that a user can refer to at their discretion.

In the above structure, the Git version control system is used to enable collaboration in GitHub. Git uses a pull-based development model in which users can keep track of changes in projects and contribute to projects by adding, editing, or evaluating content (e.g., source code). The pull-based model offers flexibility for developers to work independently by allowing them to create personal copies of projects that they can freely edit and distribute. Past research provides evidence for the benefit of this model for community engagement and project efficiency. Interestingly, this line of work also shows that developer contributions are evaluated primarily on the basis of social, rather than technical, factors (Reyes López, 2017).

Despite demonstrable success in social coding with GitHub, a number of challenges have been identified by the platform’s users. These include issues that either directly or indirectly affect teamwork and problem solving in software engineering. For example, in a recent survey of developers, GitHub users reported that they do not use certain features (e.g., the “follow” feature) because they are not useful in terms of staying up-to-date with projects. Instead, such features negatively affect their ability to use GitHub effectively by introducing information overload (Blincoe & Damian, 2015). From a theoretical standpoint, what needs to be understood is information dissemination, and user influence, in the GitHub ecosystem. That is, in order to improve the functionality of features within GitHub that support the work in which users engage in repositories, we must address a number of research questions centering on activity within repositories, and how

information propagates across repositories to influence work practice. We turn next to a discussion of how cognitive engineering can enable a richer understanding of the GitHub ecosystem and identify targets for research and development.

GITHUB AS A COMPLEX COGNITIVE ENGINEERING DOMAIN

Through an integration of theories from several disciplines, cognitive engineering can improve the study of collaborative coding in platforms like GitHub. In this section, we suggest a path forward for how cognitive engineering methods focused on team cognition (Fiore, 2012), can be used to understand, and improve, collaboration in GitHub. First, we note that team cognitive task analysis (Klein, 2000), as an analytic tool, must be employed. Second, we describe a set of collaboration concepts that can help us better understand collaboration in GitHub. For each of these, we pose representative research questions to guide the field of cognitive engineering to improve this new form of complex collaborative work.

We suggest that the cognitive engineering method of team cognitive task analysis (CTA) be utilized in the study of GitHub. In particular, team CTA (Klein, 2000) would augment our understanding by considering the collaborative and technological interactions occurring in OSSD. A team CTA can be used to assess and improve OSSD outcomes through an analysis of the team cognitive processes underlying the complex, modularized work that occurs in GitHub. For example, depending upon the nature of the distribution of the team, a team CTA can lead to the identification of the particular forms of communication that need support for effective collaboration. A team CTA would also illuminate the interdependencies within the team. This understanding could, in turn, drive the design of technology that supports coordination requirements within the project team (Fiore & Wiltshire, 2016).

In sum, our primary point is that methods from cognitive engineering, like team CTA, hold great promise for the study and improvement of OSSD. Team CTA draws attention to the interaction between cognitive processes and the components of this complex socio-technical system and can clarify issues for further research and development that improve aspects of the system. We turn next to a set of core concepts, and associated questions, that a team CTA can help us better understand.

Teamwork and Taskwork in GitHub

First, there is a pressing need to disentangle teamwork and taskwork as it occurs in GitHub. Taskwork refers to what needs to be accomplished to meet goals and complete objectives and can be broken down into knowledge, skills, and attitudes. In the context of GitHub, this would be the *knowledge* necessary for a given repository’s task (e.g., understanding relevant code). There are also *skills* needed for supporting task execution (e.g., developing and running procedures or routines). Finally, there are *attitudes* about particulars of a task one might face in a repository (e.g., preferences for methodological approaches, or trust in certain technologies).

Teamwork refers to the factors required to function effectively as collaborators and can be similarly broken down into knowledge, skills, and attitudes. In the context of GitHub, this

would be the *knowledge* associated with teammates in a repository (e.g., understanding their roles, responsibilities, and capabilities). It is also associated with the *skills* supporting interaction with teammates (e.g., communicating effectively about projects and managing conflict). Also relevant are *attitudes* about teammates based on interactions within a repository (e.g., trust in and sense of cohesion with teammates).

A team CTA is important because it can identify the collaborative and contextual features making up teamwork and taskwork in GitHub. Based upon the above distinctions, we outline the following research questions.

Research Question 1: *How do differences in taskwork and teamwork across repositories in GitHub affect project processes and outcomes?*

- Research Question 1.1. How do differences in teamwork and taskwork across repositories alter patterns of activity (e.g., distribution of activity across team members)?
- Research Question 1.2. How do differences in teamwork and taskwork across repositories change user responses to issues and other events (e.g., time to close issues)?
- Research Question 1.3. Do differences in teamwork and taskwork across repositories change the way user turnover affects activity (e.g., increase or decrease innovative ways of solving coding problems)?

Interdependencies in GitHub

Second, given the variety of projects, a team CTA can help the research community understand and better identify the differing forms of interdependencies occurring in GitHub. When looking at standard definitions of teamwork we can see a strong connection to OSSD. An early characterization of teams defined them as “interdependent collections of individuals who share responsibility for specific outcomes for their organizations” (Sundstrom, De Meuse, & Futrell, 1990, p. 120) while others noted that teams consist of “two or more individuals who must interact and adapt to achieve specified, shared, and valued objectives” (Salas, Dickinson, Converse, & Tannenbaum, 1992, p. 4). Foundational to both definitions, though, is the idea of interdependence and clear demarcation of roles and goals. We suggest that a team CTA can help identify the varying forms of interdependence within repositories to determine role relationships and coordination requirements. Following organizational distinctions of interdependence (Saavedra, Earley, & Van Dyne, 1993), team CTA can draw from this taxonomy to examine the various classes of interdependencies possible in repositories; that is, understand who relies on whom for task completion and how this alters collaboration. In brief, there are three components of complex interdependence: task, goal, and feedback interdependence.

In the interest of space, we focus on task interdependence. This characterizes the degree to which individual team members need to interact to perform the team's tasks. In its simplest form, there is pooled interdependence (Saavedre et al., 1993). In OSSD, this occurs when each developer performs his/her own task, and the team result is the sum of each developer's output. Next is sequential interdependence, which occurs when one developer's output is necessary for another developer's input (i.e., B cannot act without output from A). Third is reciprocal interdependence, where one developer's output

becomes another developer's input, and vice versa. Most complex is intensive interdependence, the highest form of coordinated activity. In OSSD, developers demonstrate this when they collaborate to jointly diagnose and solve problems (Saavedre et al., 1993). It is this form of interdependence that is likely to be the most common in repositories with ill-structured tasks, and these present the biggest socio-technical challenges for OSSD teams. In empirical studies of distributed software development, researchers have found that a shared understanding of evolving task interdependencies within a team of developers is critical for project outcomes and, more recently, introduced methods for capturing coordination requirements between developers through a network analysis of interdependent source code (Joblin, Apel, & Maurer, 2017).

We suggest that team CTA can inform the identification of meaningful qualitative differences in task interdependencies in GitHub repositories. Based upon the above distinctions, we outline the following research questions.

Research Question 2: *How does variation in interdependencies within repositories in GitHub affect project processes and outcomes?*

- Research Question 2.1. How do differences in interdependencies change information dissemination in repositories (e.g., change social network characteristics of communication)?
- Research Question 2.2. How do differences in interdependencies impact activity levels over time (e.g., alter proportion of events in a repository)?
- Research Question 2.3. Do differences in interdependencies across repositories influence activity levels of different types of users (e.g., popular versus regular users)?

Coordination Processes in GitHub

Third, and most importantly, across the variety of taskwork and teamwork, and differing forms of interdependencies, there is the need for coordination. Drawing from the organizational sciences, we suggest that the coordinative mechanisms of GitHub can be better understood through a frame of explicit and implicit coordination. Explicit coordination, like planning and communication, captures the intentional actions that individuals and groups take to identify interdependencies and formulate strategies to achieve their goals in a shared task environment. Conversely, implicit coordination captures the anticipatory and dynamic actions individuals take, without communicating, in response to an identified need (Rico, Sánchez-Manzanares, Gil, & Gibson, 2008).

An important question for team CTA is how implicit and explicit coordination manifest themselves differently in the GitHub ecosystem. In the extant literature on GitHub, researchers have recently recognized the value of understanding explicit and implicit coordinative mechanisms available through the platform. Implicit coordination has been studied in terms of the effect of team size on productivity in software development (Scholtes, Mavrodiev, & Schweitzer, 2016). Other research has explored how comments linked to user actions in GitHub serve as explicit coordinative mechanisms (Romero, Huttenlocher, & Kleinberg, 2015). Yet, most investigations of coordination in GitHub to date rely primarily on social network analysis and paint an incomplete picture of

coordination in OSSD. As such, while these investigations have provided some insights into the general functioning of the platform, they offer a shallow understanding of cognitive and social factors implicated in coordination among developers on GitHub (Yu et al., 2014). The study of coordination in GitHub is further complicated by the fact that developers rely on additional platforms and means of communication to coordinate on GitHub (e.g., Slack, Stack Overflow, face-to-face; Storey, Zagalsky, Figueira Filho, Singer, & German, 2017).

In the study of complex systems, some have looked to stigmergic processes as an explanatory mechanism that supports awareness and coordination (e.g., Christensen, 2008). Stigmergy, a term that emerged from the biological sciences to describe bottom-up mechanisms producing complex, emergent behavior in social insects, is the result of tangible objects (from pheromone trails in insects, to document notations in organizations), that act as coordinative mechanisms to influence individual behavior without planning or centralized control. Specifically, an agent is compelled to act, usually to complete a task, in response to a stimulus left in the environment by another agent (Theraulaz & Bonabeau, 1999). Stigmergy has been adapted in a number of domains to explain coordinative behavior; more recently, it has been integrated with organizational theories of implicit coordination in OSSD (Howison, Østerlund, Crowston, & Bolici, 2012), as well as in GitHub (Goggins et al., 2013; Blincoe & Damian, 2015).

A team CTA can help uncover the varied forms of explicit and implicit coordination in GitHub. Based on the above distinctions, we outline the following research questions.

Research Question 3: *How can team CTA inform our understanding of explicit and implicit coordination mechanisms in GitHub repositories?*

- Research Question 3.1. Does implicit or explicit coordination determine how users contribute to repositories (e.g., more contributions when explicit coordination is present)?
- Research Question 3.2. Does the degree of explicit versus implicit coordination determine whether community activity is static or occurs in bursts (e.g., changes in proportion of coordination types change activity dynamics)?
- Research Question 3.3. Does the proportion of explicit versus implicit coordination change how issues and other events are addressed by users (e.g., make concrete the collaboration need, thus influencing efficiency)?

Awareness and Influence in GitHub

Early studies of computer-supported collaboration described how communities build and share knowledge via activity awareness. This was shown to be critical to influence in the community in that the distributed nature of the work, in conjunction with a lack of co-location, hindered member ability to assess what has been and what needs to be done (Gutwin & Greenberg, 2002). Others have noted that awareness is not only needed but can also be augmented such that knowledge production is amplified (Carroll et al., 2009). For example, distributed communities can develop and collectively draw from shared repositories of knowledge to create an extended cognitive system (e.g., Mendeley scholarly database).

On GitHub, awareness is, at least to a certain extent, a function of the transparency of the pull-based development

model the platform uses to track the contributions of multiple developers in a project. Developers may engage in explicit communication when GitHub's structure fails to support the awareness of information necessary for a given task (Dabbish, Stuart, Tsay, & Herbsleb, 2012). Further, as previously mentioned, developers have reported that some GitHub features produce an unmanageable amount of information that is ultimately detrimental to awareness and, in turn, implicit coordination (Blincoe & Damian, 2015).

Within this social coding platform, influence directly speaks to user activity and the awareness that users have of other users' activity. In order to be influenced, a user must be aware of what another user has done with respect to user-repository interaction (e.g., contributing source code). Research shows that users follow rock stars (users who amass a large following on GitHub) and are thus aware of what rock stars are doing (Dabbish et al., 2012). For example, when a rock star stars a repository, their followers will subsequently star the same repository without a resulting task activity related to that event (Blincoe et al., 2016). This study also found that a rock star's followers can be influenced to contribute to a repository that the rock star has recently starred. Rock stars produce a similar effect in terms of influence when they create a copy of another user's repository (Blincoe & Damian, 2015).

Given these findings, and the interactions afforded by GitHub, we can attempt to understand influence as either explicit or implicit. For instance, on one hand, we can conceptualize implicit influence as the degree to which a user mimics the behavior of a rock star. Explicit influence, on the other hand, may result from a user action in which they create a comment about something a rock star said or did. We suggest that team CTA can be used to uncover how influence and awareness varies across GitHub repositories to alter activity and outcomes. Based upon the above distinctions, we outline the following research questions.

Research Question 4: *What are the features within GitHub affecting the degree of awareness and influence found across repositories?*

- Research Question 4.1. Do differences in numbers of rock stars associated with a repository change how influence propagates across users (e.g., proportional influence is lower with more rock stars)?
- Research Question 4.2. How does awareness in GitHub determine the speed with which repositories become popular (e.g., network dynamics influence rate of popularity)?
- Research Question 4.3. How do differences in the use of interaction features versus social network features explain awareness of activity and user influence in GitHub (e.g., submitting a pull request versus following a user)?

CONCLUSIONS

GitHub is a relatively new problem-solving system that requires effective coordination for the emergence of applied solutions and general knowledge (Eccles & Groth, 2007). The structure of OSSD requires developers to adapt to the technological constraints governing their actions and interactions with each other. Through use of cognitive engineering meth-

ods, it is possible to augment developer and technological capabilities to support complex collaborative problem solving.

With this novel problem-solving system, we suggest that a closer examination of the social and task components of OSSD as they relate to GitHub is essential to effective coordination. Research must seek to understand how users collaborate and what trade-offs exist in GitHub with respect to problem solving effectiveness; for example, how communication alters coordination (Dabbish et al., 2012). Additionally, understanding developer characteristics (e.g., trust) that influence users' propensity to accept contributions from other users, is critical for the effective use of GitHub for problem solving (Soto, Coker, & Le Goues, 2017). Finally, we contend that popularity and influence are important factors for achieving awareness and facilitating coordination for OSSD in GitHub.

In sum, we offer a perspective for understanding and improving coordination in OSSD on GitHub. With the increasing prevalence of the social coding paradigm and OSSD, the importance of effective and efficient coordination in distributed, asynchronous software development is undeniable. Cognitive engineering can use platforms like GitHub to explore the effect of social features on coordination in socio-technical problem-solving systems. Through an integration of theory and research from the organizational sciences with the extant literature on social coding in GitHub, we have delineated a preliminary set of research questions for future research aimed at improving our understanding of collaboration in OSSD.

ACKNOWLEDGEMENTS

This work was partially supported by grant FA8650-18-C-7823 from the Defense Advanced Research Projects Agency (DARPA). The views and opinions contained in this article are the authors and should not be construed as official or as reflecting the views of the University of Central Florida, DARPA, or the U.S. Department of Defense.

REFERENCES

- Abrahamsson, P., Salo, O., Ronkainen, J., & Warsta, J. (2002). *Agile Software Development Methods: Review and Analysis*. Espoo, Finland: Otava-media Oy.
- Blincoe, K., & Damian, D. (2015). Implicit coordination: A case study of the rails OSS project. In *IFIP International Conference on Open Source Systems* (pp. 35–44). Cham, Switzerland: Springer.
- Blincoe, K., Sheoran, J., Goggins, S., Petakovic, E., & Damian, D. (2016). Understanding the popular users: Following, affiliation influence and leadership on GitHub. *Information and Software Technology*, 70, 30–39.
- Carroll, J. M., Rosson, M. B., Farooq, U., & Xiao, L. (2009). Beyond being aware. *Information and Organization*, 19(3), 162–185.
- Christensen, L. R. (2008). The logic of practices of stigmergy: Representational artifacts in architectural design. In *Proceedings of the 2008 ACM Conference on Computer Supported Cooperative Work*, (pp. 559–568). New York, NY: ACM Press.
- Dabbish, L., Stuart, C., Tsay, J., & Herbsleb, J. (2012). Social coding in GitHub: Transparency and collaboration in an open software repository. In *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work* (pp. 1277–1286). ACM Press.
- Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. *Journal of Systems and Software*, 85(6), 1213–1221.
- Eccles, D. W., & Groth, P. T. (2007). Wolves, bees, and football: Enhancing coordination in sociotechnological problem solving systems through the study of human and animal groups. *Computers in Human Behavior*, 23(6), 2778–2790.
- Fiore, S. M. (2012). Cognition and technology: Interdisciplinarity and the impact of cognitive engineering research on organizational productivity. In S. Kozlowski (Ed.). *Oxford Handbook of Industrial and Organizational Psychology* (pp. 1306–1322). Oxford University Press.
- Fiore, S. M., Carter, D. R., & Asencio, R. (2015). Conflict, trust, and cohesion: Examining affective and attitudinal factors in science teams. In E. Salas, W.B. Vessey, & A.X. Estrada (Eds.), *Team Cohesion: Advances in Psychological Theory, Methods and Practice* (pp. 271–301). Emerald Group Publishing Limited.
- Fiore, S. M., Rosen, M. A., Smith-Jentsch, K. A., Salas, E., Letsky, M. & Warner, N. (2010). Toward an understanding of macrocognition in teams: Predicting processes in complex collaborative contexts. *Human Factors*, 52, 2, 203–224.
- Fiore, S. M. & Wiltshire, T. J. (2016). Technology as teammate: Examining the role of external cognition in support of team cognitive processes. *Frontiers in Psychology: Cognitive Science*. 7:1531.
- Goggins, S. P., Mascaro, C., & Valetto, G. (2013). Group informatics: A methodological approach and ontology for sociotechnical group research. *Journal of the Association for Information Science and Technology*, 64(3), 516–539.
- Gutwin, C., & Greenberg, S. (2002). A descriptive framework of workspace awareness for real-time groupware. *Computer Supported Cooperative Work*, 11, 411–446.
- Howison, J., Østerlund, C., Crowston, K., & Bolici, F. (2012). Stigmergy and implicit coordination in software development. In *CSCW '12* (Vol. 12, pp. 11–15). New York, NY: ACM Press.
- Joblin, M., Apel, S., & Maurer, W. (2017). Evolutionary trends of developer coordination: A network approach. *Empirical Software Engineering*, 22(4), 2050–2094.
- Klein, G. (2000). Cognitive task analysis of teams. In J. M. Schraagen, S. F. Chipman, & V. L. Shalin, (Eds.) *Cognitive Task Analysis* (pp. 417–430). Mahwah, NJ: Erlbaum.
- McDonald, N., & Goggins, S. (2013). Performance and participation in open source software on GitHub. In *CHI '13 Extended Abstracts on Human Factors in Computing Systems* (pp. 139–144). New York, NY: ACM Press.
- Octoverse 2017: Highlights from the last 12 months. (2018). Retrieved from <https://octoverse.github.com>
- Reyes López, A. (2017). *Analyzing GitHub as a Collaborative Software Development Platform: A Systematic Review* (Masters). University of Victoria, British Columbia, Canada.
- Rico, R., Sánchez-Manzanares, M., Gil, F., & Gibson, C. (2008). Team implicit coordination processes: A team knowledge-based approach. *Academy of Management Review*, 33(1), 163–184.
- Romero, D. M., Huttenlocher, D., & Kleinberg, J. M. (2015). Coordination and efficiency in decentralized collaboration. In *Proceedings of the 9th International AAAI Conference on Web and Social Media* (pp. 367–376). AAAI Press.
- Saavedra, R. P., Earley, P. C., & Van Dyne, L. (1993). Complex interdependence in task-performing groups. *Journal of Applied Psychology*, 78(1), 61–72.
- Salas, E., Dickinson, T. L., Converse, S. A., & Tannenbaum, S. I. (1992). Toward an understanding of team performance and training. In R. W. Swezey, E. Salas, (Eds.), *Teams: Their Training and Performance* (pp. 3–29). Westport, CT: Ablex Publishing.
- Scholtes, I., Mavrodiev, P., & Schweitzer, F. (2016). From Aristotle to Ringelmann: A large-scale analysis of team productivity and coordination in Open Source Software projects. *Empirical Software Engineering*, 21(2), 642–683.
- Soto, M., Coker, Z., & Le Goues, C. (2017). Analyzing the impact of social attributes on commit integration success. In *Proceedings of the 14th International Conference on Mining Software Repositories* (pp. 483–486). IEEE Press.
- Storey, M. A., Zagalsky, A., Figueira Filho, F., Singer, L., & German, D. M. (2017). How social and communication channels shape and challenge a participatory culture in software development. *IEEE Transactions on Software Engineering*, 43(2), 185–204.
- Sundstrom, E., De Meuse, K. P., & Futrell, D. (1990). Work teams: Applications and effectiveness. *American Psychologist*, 45(2), 120–133.
- Theraulaz, G., & Bonabeau, E. (1999). A brief history of stigmergy. *Artificial Life*, 5(2), 97–116.
- Yu, Y., Yin, G., Wang, H., & Wang, T. (2014). Exploring the patterns of social behavior in GitHub. In *Proceedings of the 1st International Workshop on Crowd-based Software Development Methods and Technologies* (pp. 31–36). New York, NY: ACM Press.